Web Programming - Assignment Report

Athanasios Argyropoulos

E22012

Table of Contents

Table of Contents 3
Chapter 1: Introduction 4
Chapter 2: System Description 5
[2.1]: Management Subsystem 5
[2.2]: User Subsystem 6
Chapter 3: System Implementation 7
[3.1]: Tech Stack 7
[3.2]: Database Schema 7
[3.3]: REST APIs 9
Chapter 4: Administrator Manual 19
[4.1]: Prerequisites 19
[4.2]: Installation 19
Chapter 5: User Manual 20
[5.1]: User 20
[5.2]: Administrator 26
[5.3]: Technician 32

Chapter 1: Introduction

The following project is a university assignment with the purpose of helping the student get familiarized with the basic architectures and technologies used in the scope of network-centered Informational System development.

More specifically, the assignment demands the development of an Informational System for a company's management of technical support requests. This assignment is to be developed with either mySQL or noSQL as the database, while the Backend and Frontend technologies are up to the student's decision.

The student is also required to handle the use of external public APIs in their project implementation, while also maintain proper error handling and user friendliness.

Chapter 2: System Description

The system will support the logging, monitoring, and resolution of technical support requests and will consist of two subsystems: the management subsystem (administrators / technicians) and the user subsystem. Specifically, it will support the following:

[2.1]: Management Subsystem

1) Management of system registration requests:
The system administrator will have access to requests for registration in the system and will be able to approve or reject them. The user registration process is described in detail below.

2) Management of system users:
The system administrator will have access to the system's users and will be responsible for updating their information in the database (Create, Read, Update, Delete – CRUD operations). The administrator will also be able to assign or modify a user's role (e.g., regular user, technician, administrator).

3) Management of technical support request categories:
The administrator will manage problem categories, such as, indicatively, software issues, hardware issues, system access, network/connectivity issues. A priority level will be defined for each category.

4) Management of technical support requests:
The administrator or technician will be able to view all requests, assign requests to technicians, update the status of a request (e.g., new, in progress, completed), and record comments and resolution actions.

5) Management of system announcements:
The administrator will be able to post announcements (e.g., scheduled maintenance, service interruptions), which will be visible to all registered users.

[2.2]: User Subsystem

1) Submission of a technical support request:
A registered user will be able to submit a new technical support request, select a problem category, describe the issue, and attach files (e.g., screenshots).

2) Monitoring request status:
The user will be able to view the list of requests they have submitted and their current status.

3) Request history:
The user will have access to the complete history of their requests, including completed ones.

4) Announcements:
The user will be able to view all announcements posted by the management.

To register users in the system, the following steps must be followed:
Any user who wishes to use the system must be registered. To register, a user must complete a form with their details, which will include, at a minimum, first name, last name, country, city, address, e-mail, username, and password. Subsequently, the system administrator will review the registration request and, if it is approved, will assign a role to the user (regular user, technician, or administrator). Depending on the role assigned, the user gains access to the corresponding subsystem of the information system.

Chapter 3: System Implementation

[3.1]: Tech Stack

The system meant to meet the goals of this assignment was created using the following tech stack:

• ***Backend:*** Java. More specifically, Spring Boot was used in order to implement the server and API endpoints.

• ***Frontend:*** Typescript. More specifically, Typescript in combination with the React library and Tailwind CSS.

• ***Database:*** mySQL. More specifically, mySQL database provided through the XAMPP Control Panel.

[3.2]: Database Schema

The database schema that the implemented project needs to function is the following:

User:

Field	Type	Purpose
id	int	An auto-incremented identifier for the entity.
email	varchar	The user's email.
password	varchar	The user's **hash** of their password.
name	varchar	The user's name.
surname	varchar	The user's surname.
country	varchar	The user's country.
city	varchar	The user's city.
address	varchar	The user's address.
username	varchar	The user's username.
role	varchar	The user's role in the system: USER, ADMIN, TECHNICIAN, NOT_DETERMINED

Categories

Field	Type	Purpose
id	int	An auto-incremented identifier for the entity.
name	varchar	The category's name.
description	varchar	The category's description.
priority	varchar	The category's priority.

Requests

Field	Type	Purpose
id	bigint	An auto-incremented identifier for the entity.
description	varchar	The request's description.
category_id	int	The request's category represented its id.
status	varchar	The request's status: NEW, IN_PROGRESS, COMPLETED
created_at	timestamp	The request's time of creation.
updated_at	timestamp	The request's time of latest update.
user_id	int	The request's user represented by their id.
technician	int	The request's assigned technician.
comments	varchar	The request's comments.
actions	varchar	The request's actions.

Request_files

Field	Type	Purpose
id	bigint	An auto-incremented identifier for the entity.
filename	varchar	The file's name.
path	varchar	The file's path in the server's storage.
request_id	bigint	The file's request represented by its id.

Announcements

Field	Type	Purpose
id	int	An auto-incremented identifier for the entity.
title	varchar	The announcement's title.
content	varchar	The announcement's content.
created_at	timestamp	The announcement's creation date.

[3.3]: REST APIs

- Important to note that All REST API endpoints in the server start with the prefix:

http://localhost:8080/api

- Also, in the parameters, `parameter: {Type property}` refers to a Java object. The Frontend can send a regular JSON with the same properties and Spring will automatically convert it into the specified Java object. So, the parameter is a Java object, but it is not expected for the frontend to send an actual Java object, just a JSON with the same structure.

Endpoint:	**Method:**
/auth/login	POST
Functionality	
Check given credentials. Create a JWT if they are correct or respond with error.	
Parameters	
Login Request: {String email, String password} *HTTP Response*	
Headers:	
Cookies: required *Content-type*: JSON	
Body:	
{email, password}	

Endpoint:	**Method:**
/auth/logout	POST
Functionality	
Nullify the JWT's value and expire the cookie.	
Parameters	
HTTP Response	
Headers:	
Cookies: required *Content-type:* JSON	
Body:	
none	

Endpoint:	**Method:**
/auth/register	POST
Functionality	
Check if the user is logged in. If not, create a new user with the provided information.	
Parameters	
Register Request: {String name, String surname, String country, String city, String address, String username, String email, String password} *Authentication:* An internal Spring Bean for authentication	
Headers:	
Content-type: JSON	
Body:	
{name, surname, country, city, address, username, email, password}	

Endpoint:	**Method:**
/auth/check	GET
Functionality	
Check if the user is logged in. If yes, return their role, otherwise, a message.	
Parameters	
JWT	
Headers:	
Cookies: required	
Body:	
none	

Endpoint:	**Method:**
/announcements	POST
Functionality	
Create a new Announcement	
Parameters	
Announcement: {String title, String content}	
Headers:	
Cookies: required *Content-type:* JSON	
Body:	
{title, content}	

Endpoint:	**Method:**
/announcements	GET
Functionality	
Get all announcements from the database.	
Parameters	
none	
Headers:	
Cookies: required	
Body:	
none	

Endpoint:	**Method:**
/categories	GET
Functionality	
Get all categories from the database.	
Parameters	
none	
Headers:	
none	
Body:	
none	

Endpoint:	**Method:**
/categories	PUT
Functionality	
Update the information of a current category.	
Parameters	
CategoryDTO: {int id, String name, String description, String priority}	
Headers:	
Cookies: required Content-type: JSON	
Body:	
{id, name, description, priority}	

Endpoint:	**Method:**
/category/{id}	DELETE
Functionality	
Delete a currently existing category.	
Parameters	
Id: The path parameter that represents the category's id.	
Headers:	
Cookies: required	
Body:	
none	

Endpoint:	**Method:**
/categories	POST
Functionality	
Create a new Category.	
Parameters	
CategoryDTO: {int id, String name, String description, String priority}	
Headers:	
Cookies: required *Content-type:* JSON	
Body:	
{name, surname, country, city, address, username, email, password}	

Endpoint:	**Method:**
/requests	POST
Functionality	
Create a new request.	
Parameters	
categoryId: The id of the category that was selected, *Description:* The description for the request, *Files:* Files submitted during the request, *Token:* JWT	
Headers:	
Cookies: required *Content-type:* multipart/form-data	
Body:	
{categoryId, description, files}	

Endpoint:	**Method:**
/requests	GET
Functionality	
Retrieve all system requests or all requests for a user or all request for a user except the ones with a specific status.	
Parameters	
Authentication: An internal Spring Bean for authentication, *excludeStatus*: A query parameter representing a value from the enumeration: {NEW, IN_PROGRESS, COMPLETED}, *all:* A boolean value to indicate all requests	
Headers:	
Cookies: required	
Body:	
none	

Endpoint:	**Method:**
/requests	PUT
Functionality	
Update the information of a request.	
Parameters	
Update Request: {Integer id, String status, String actions, String comments, Integer, technician}	
Headers:	
Cookies: required *Content-type:* JSON	
Body:	
{id, status, actions, comments, technician}	

Endpoint:	**Method:**
/files/{id}	GET
Functionality	
Return a specific file based on the given id.	
Parameters	
Long Id: path parameter representing the desired file's id *String token:* JWT	
Headers:	
none	
Body:	
none	

Endpoint:	**Method:**
/users	GET
Functionality	
Return user by id or users by role.	
Parameters	
Authentication: An internal Spring Bean for authentication, *Role:* The query parameter that represents the role to search by: { USER, TECHNICIAN, ADMIN, NOT_DETERMINED } *Id:* The query parameter that represents the desired user's id.	
Headers:	
Cookies: required	
Body:	
none	

Endpoint:	**Method:**
/users/role	PUT
Functionality	
Update the role of a User.	
Parameters	
UpdateRoleRequest: {Integer id, String role}	
Headers:	
Cookies: required *Content-type:* JSON	
Body:	
{id, role}	

Endpoint:	**Method:**
/users/user	PUT
Functionality	
Update all of the User's information.	
Parameters	
UserDTO: {int id, String name, String surname, String country, String city, String address, String username, String email, String role}	
Headers:	
Cookies: required *Content-type:* JSON	
Body:	
{id, name, surname, country, city, address, username, email, role}	

Endpoint:	**Method:**
/{id}	DELETE
Functionality	
Delete a user based on the given id.	
Parameters	
Integer id: A path parameter that represents the user's id.	
Headers:	
Cookies: required	
Body:	
none	

Endpoint:	**Method:**
/ or /path/	GET
Functionality	
Render the index.html	
Parameters	
none	
Headers:	
none	
Body:	
none	

Chapter 4: Administrator Manual

[4.1]: Prerequisites

- An IDE like VScode, IntelliJ, Eclipse or a Command Prompt.
- Recommended at least Maven-3.9.12 and Java 25.0.1
- XAMPP Control Panel

[4.2]: Installation

In the folder containing this file, the Administrator can find the `reactserver` folder as well as the `mihelper_db.sql` file.

Database:

1) Open the XAMPP Control Panel, and click start for the Apache and MySQL modules (Must remain active while the server is active as well).

2) After they have loaded, click on the `Admin` button of the MySQL module, so that you can access your local phpmyadmin page.

3) On the sidebar of the page, click `new` to create a new database with the name: mihelper_db.

4) Enter your new database's section and click on the `Import` tab. There you can import the `mihelper_db.sql` file and click the `Import` button at the end.

Server using IDE:

1) Load your IDE of choice, and inside open the `reactserver` folder.
2) Initiate the application.

Server using CMD:

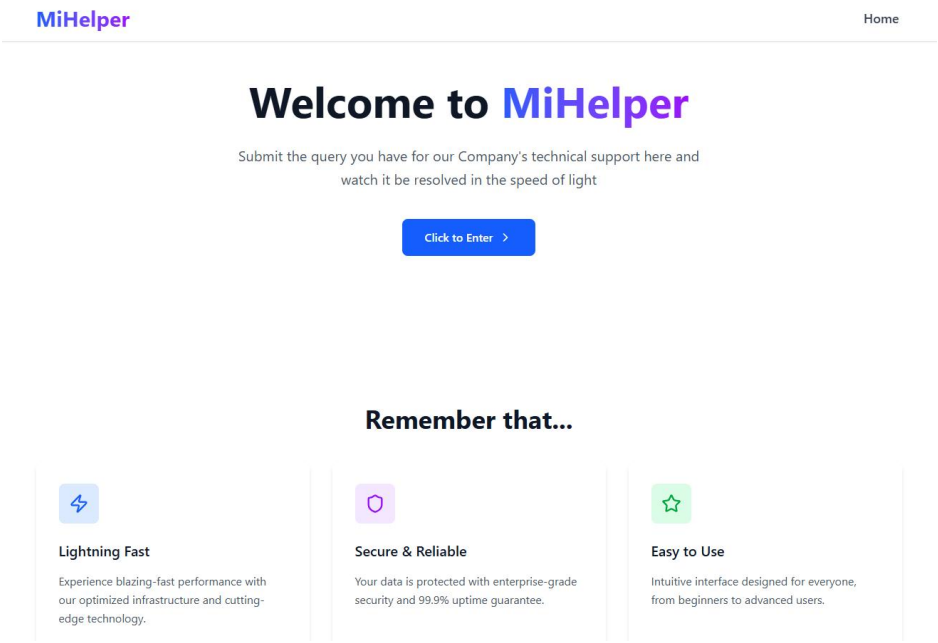
1) Open a CMD on the `reactserver` folder and run the command:
`mvn spring-boot:run`.

You can now access the website on `http://localhost:8080/`.

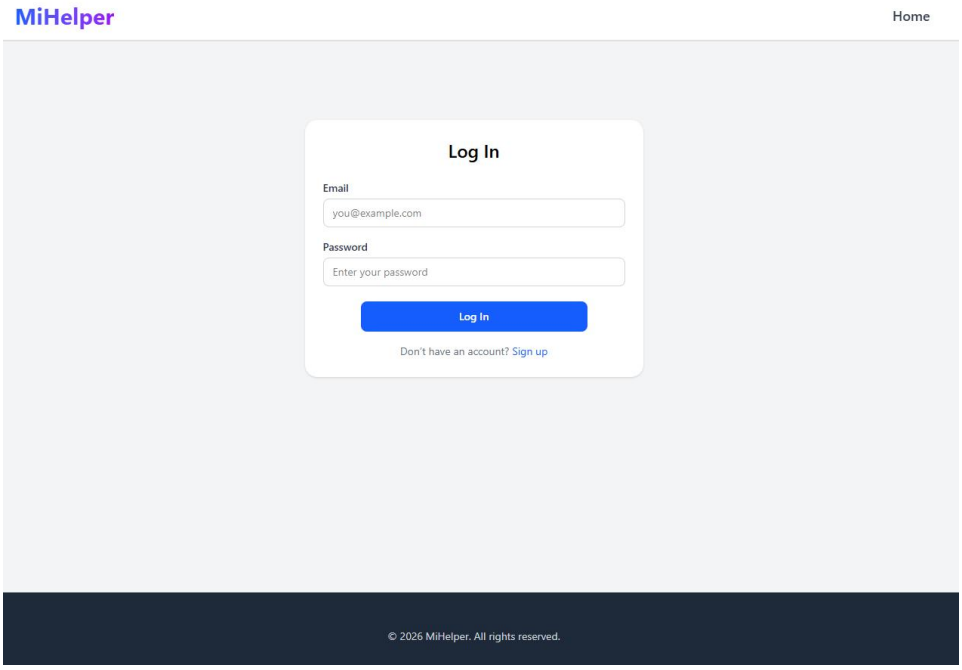
Chapter 5: User Manual

[5.1]: User

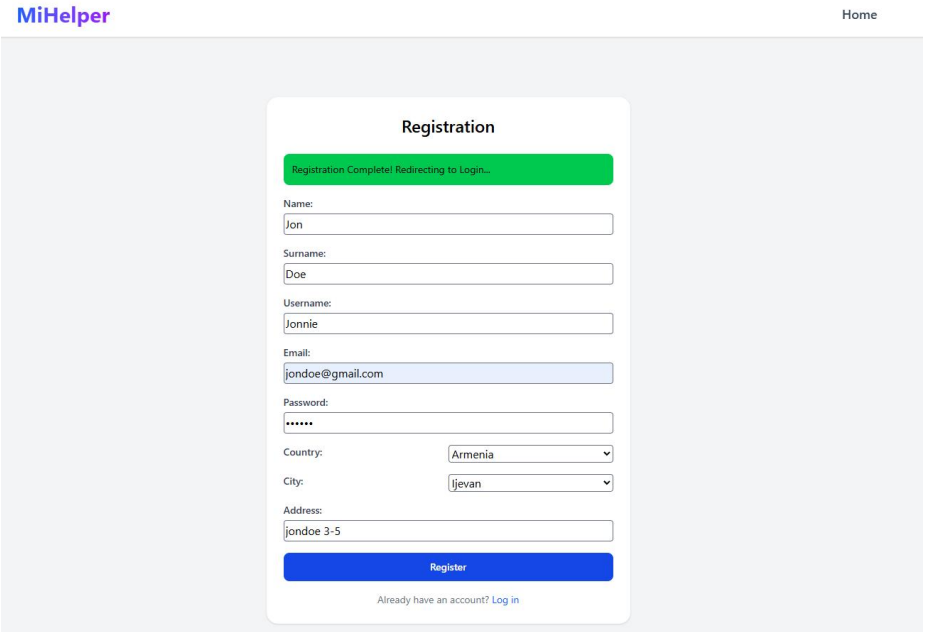
When the user first encounters the website, they will see the intro:



Here the user can get a good understanding of the website's capabilities as well as an easy access to the login form through the `Click to Enter` button:



Here the user can enter their credentials: email and password, that were submitted during their registration in order to enter the system. If the current user is not already registered, then they can click the link below the `Log In` button to get transferred to the Registration page:



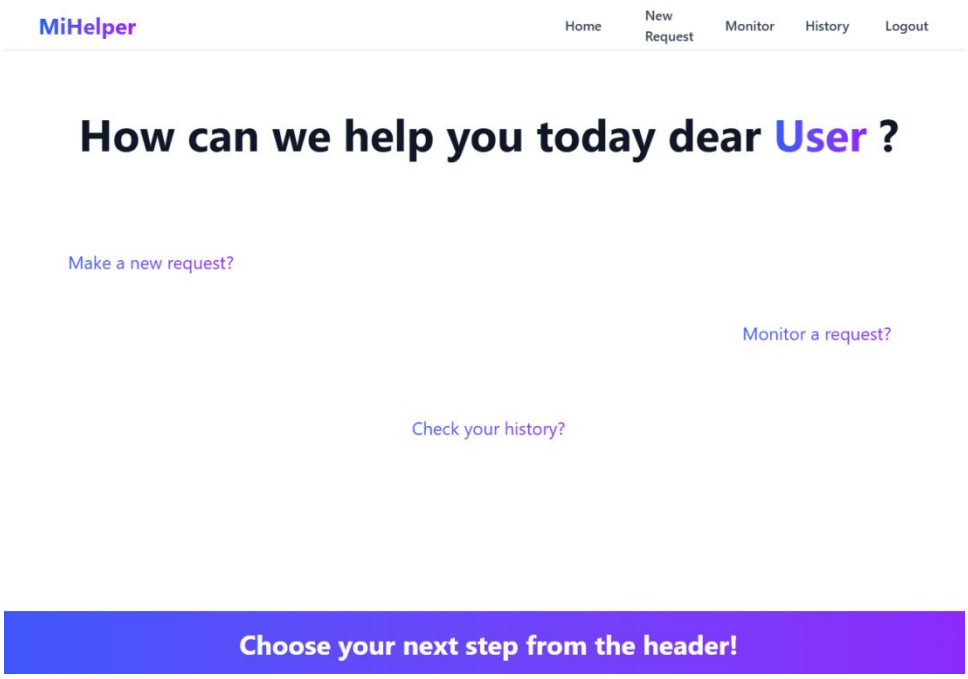
Once here, the user can enter information, which includes:

- Name
- Surname
- Username
- Email
- Password
- Country
- City
- Address

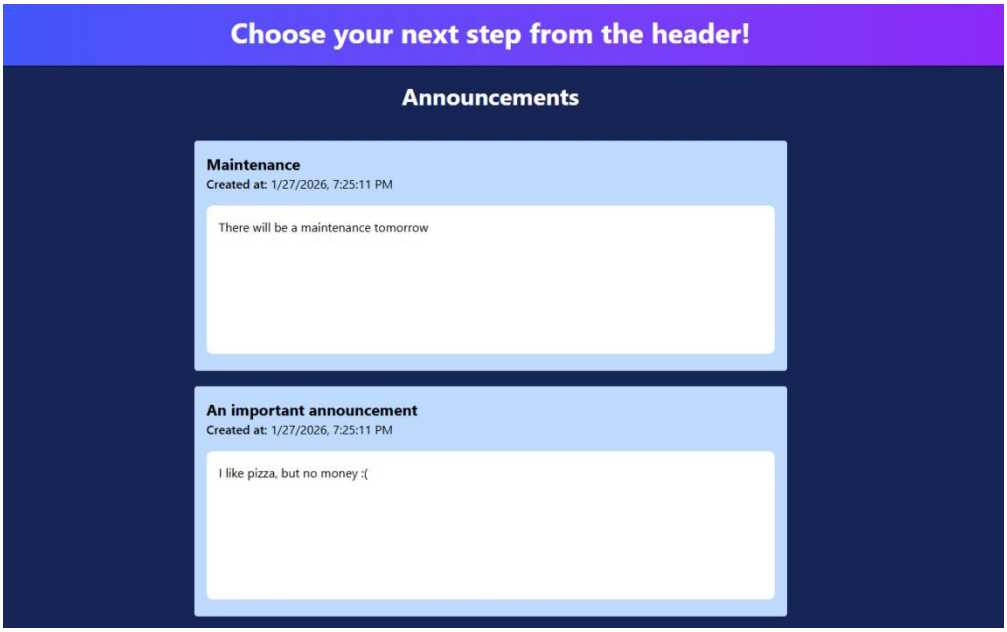
After completing the form, the user can click the `Register` button in order to complete the registration. If all goes well, a message will be shown to inform the user and that they will be transferred to the Login page. Once there, they can enter their credentials and login to the system.

However, there is a high possibility that the Administrator has not yet assigned them a role or accepted their request for registration, thus, the user will be shown a different page from the normal Home for login users once they login in order to inform them of such case:

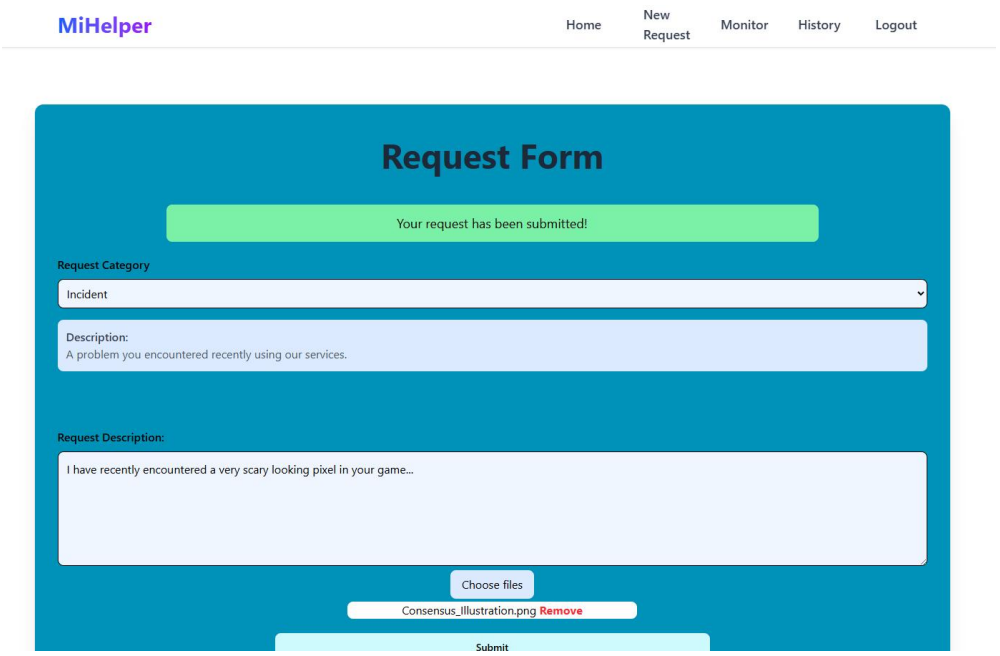
Once the Administrator has accepted their registration, the User will be able to enter the original Home for logged in users:



Here, the User can choose their next step from the header, as well as be informed about any announcement the Administrator has made in the last 30 days:



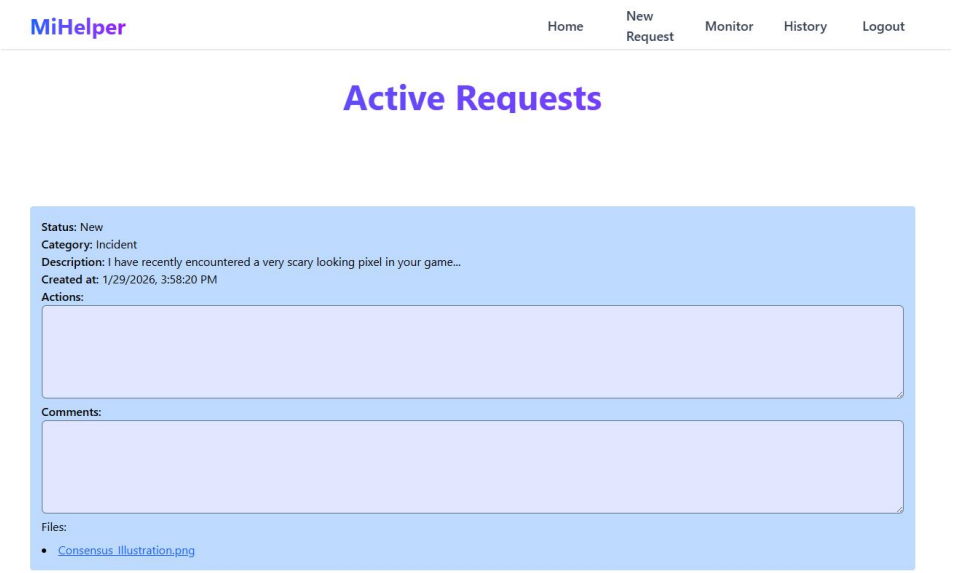
For starters, a user is able to create a new technical support request for the company by clicking the `New Request` button in the navigation bar, which will transfer them to the Request form for them to fill out:



Here, the user can choose the type of request they have through the drop down. A small description of the request type is always shown below so that the user can decide more easily which type suits their case the best. Additionally, they are able to describe their problem/request in the text box given, as well as input various files using the `Choose files` button.

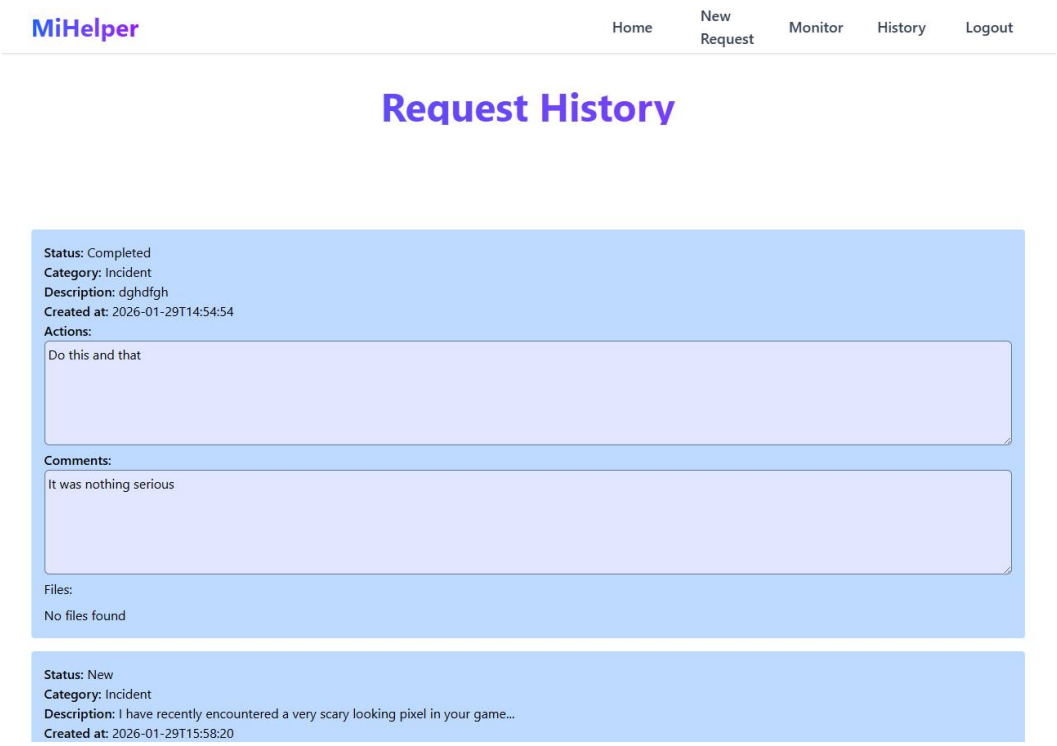
When the user is satisfied, they can click the `Submit` button in order to save their request to the system. A small message will also be shown to inform them of the result.

A user is also able to keep track of their requests by clicking the `Monitor` button in the navigation bar, which will transfer them to their Active Requests Page:



Here, the user can observe every non-completed request of theirs, including the current state: new or in progress, the information and files they submitted during its creation, the day it was created, the possible actions to solve the problem, as well as comments by the technicians/Administrator.

The user is also able to check their complete history of their request by clicking the `History` button in the navigation bar, which will transfer them to the History page:

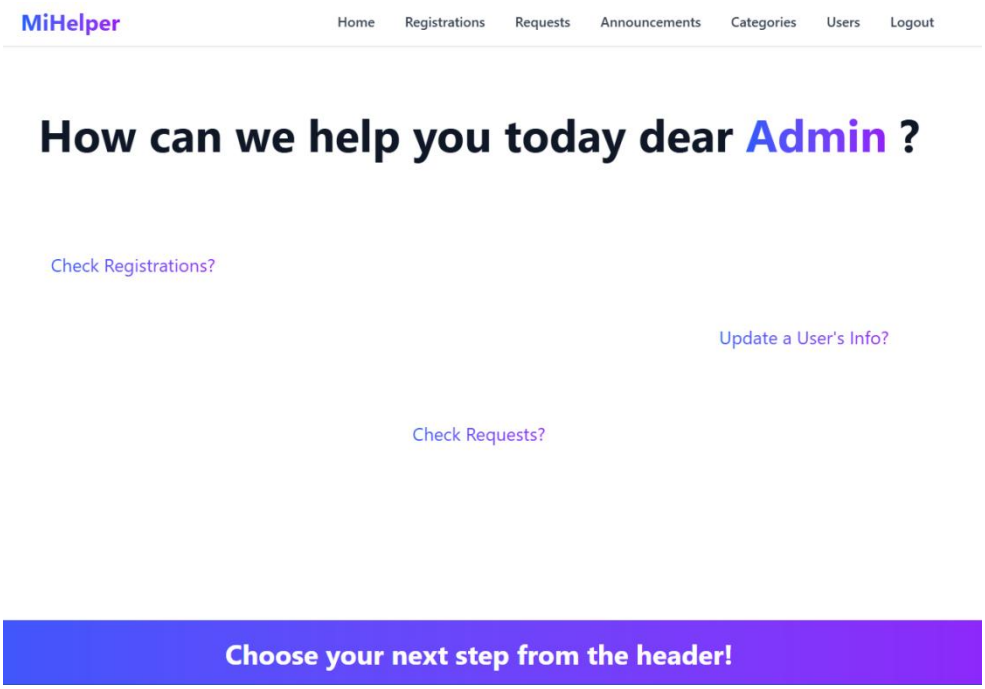


Here, the user can keep track of all of their requests with the company, even those that have already been completed.

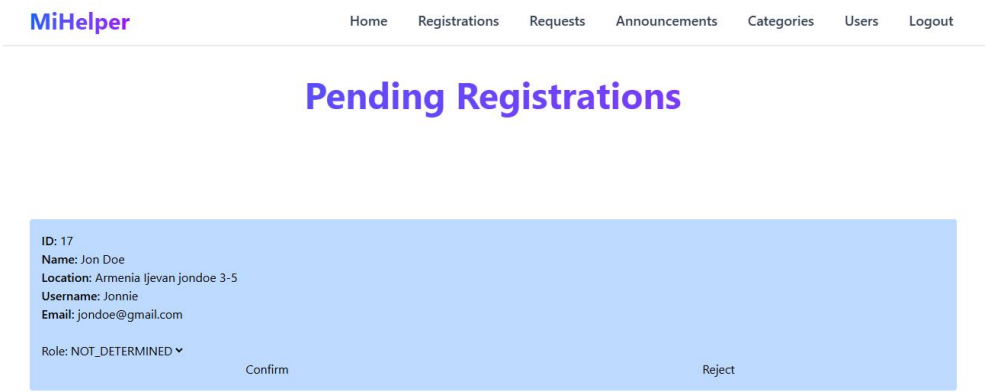
Lastly, the User is able to logout from the site by clicking the `Logout` button on the navigation bar.

[5.2]: Administrator

Once the Administrator logs in with their credentials (have already been added by default in the database). They will be shown the Administrator Home page:

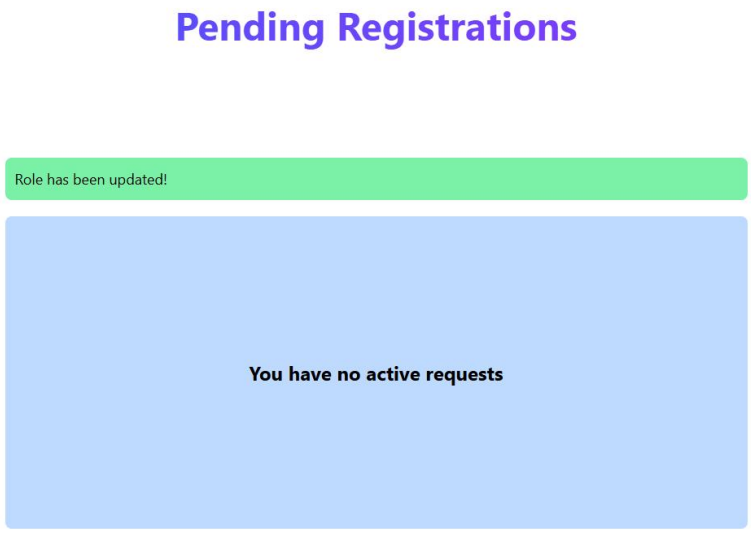


Where they can also see the announcements below, just like the user and is able to choose their next step from the header. For starters, they are able to check any registration requests by clicking the `Registrations` button in the navigation bar, which will transfer them to the Registrations page:

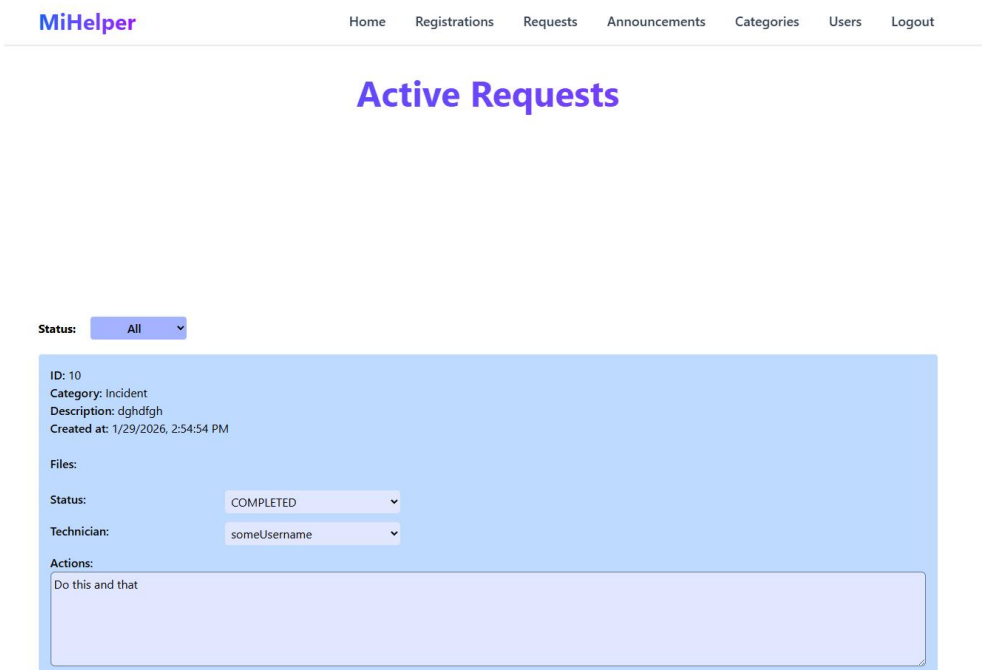


Here, the Admin can choose whether they want to set a role to the incoming user or just reject their request. The Admin can choose a role from the drop down: USER, TECHNICIAN, ADMINISTRATOR and click the `Confirm` button to save their role.

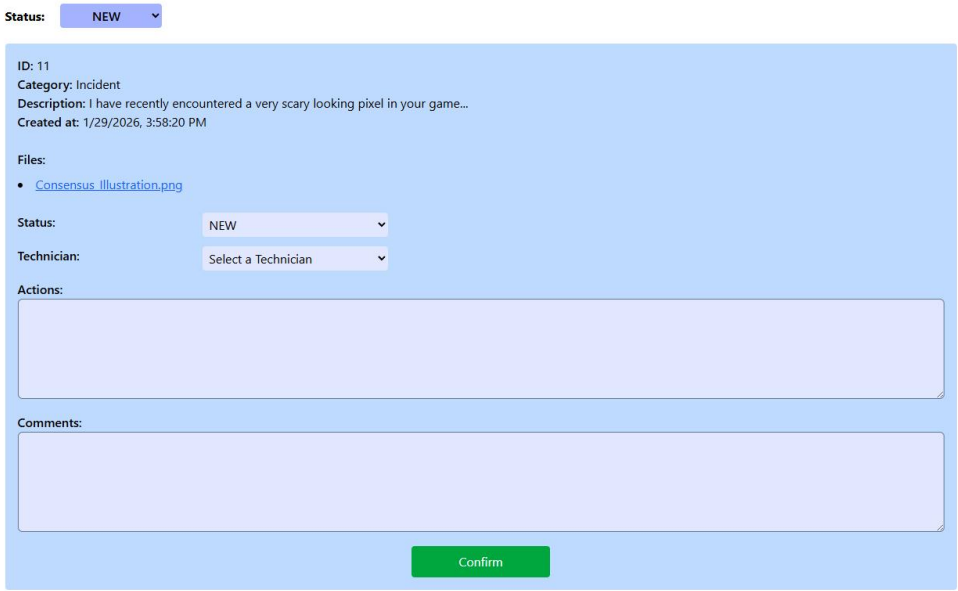
To reject the request, they need to click on the `Reject` button. In either case, a message will be displayed to inform the Admin of the result:



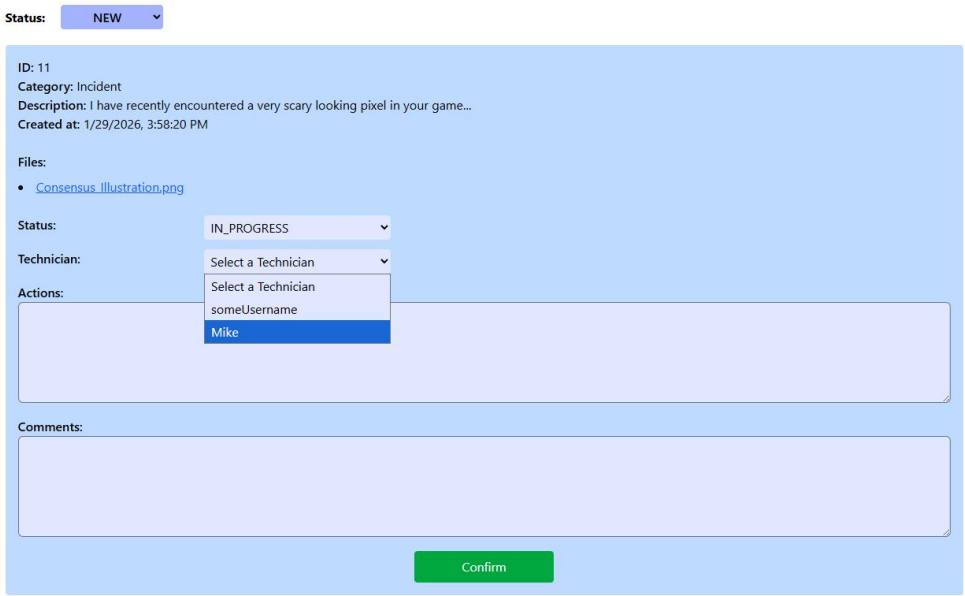
The Admin is also able to check all requests in the system by clicking the `Requests` button in the navigation bar, which will transfer them to the Requests page:



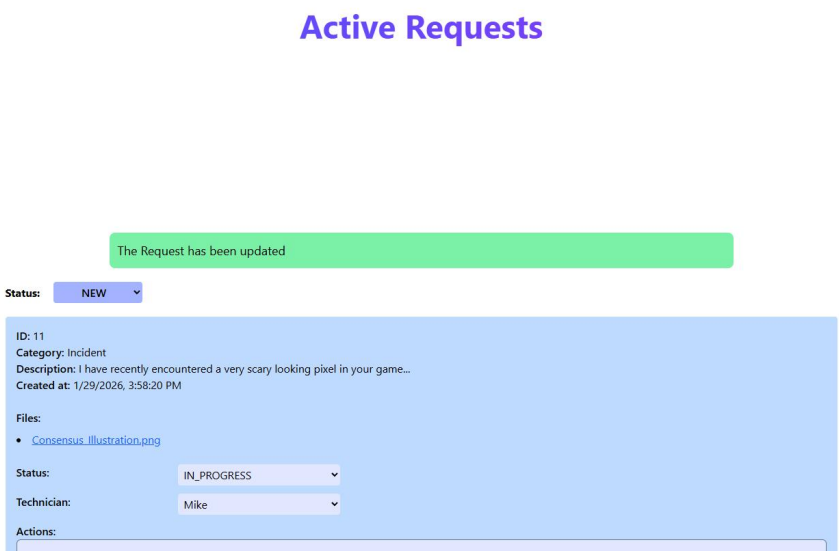
Here, the Admin can filter the displayed requests by their saved status, which can be either new, in progress or completed:



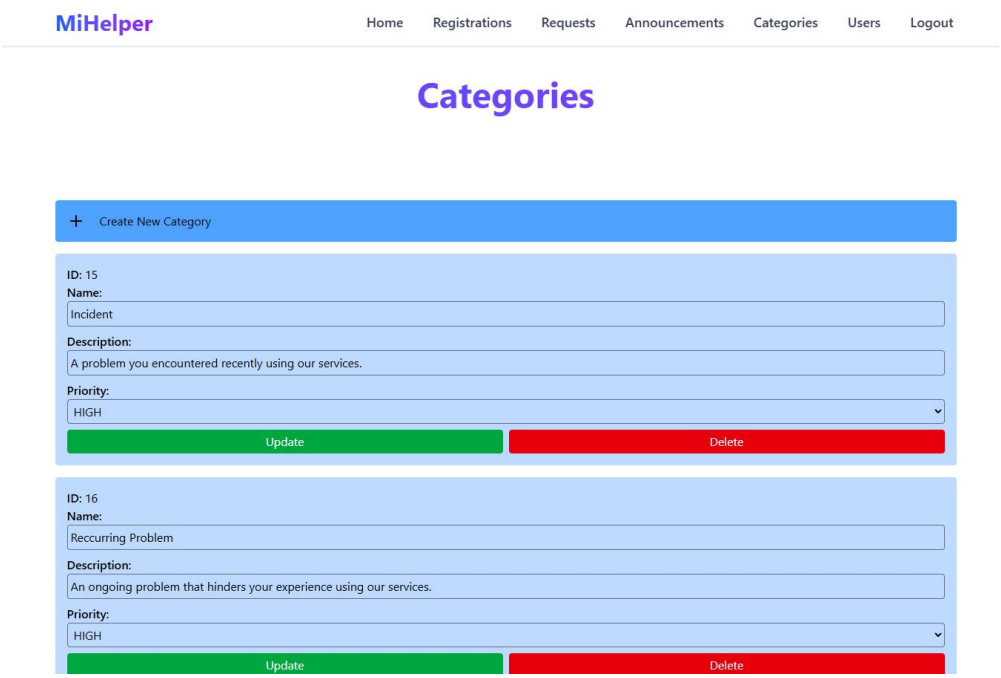
They can also update the status, their assigned Technician, the actions and the comments. They are also able to download any file the request was submitted with.



Whenever a request is updated, the page will scroll to the top and a message will be shown to inform the Admin of the result.



The Admin is also able to manage the system's categories by clicking the `Categories` button in the navigation bar, which will take them to the Categories Management page:

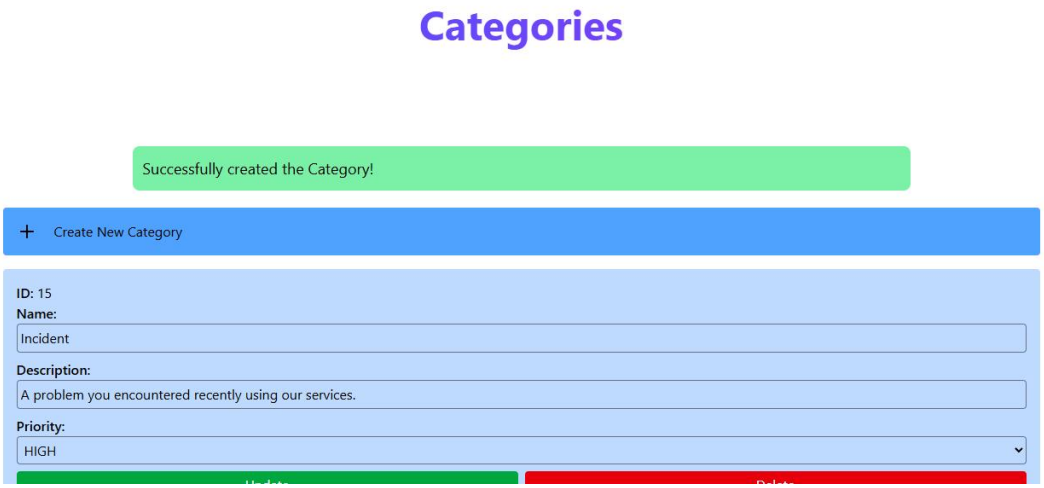


Here, the User is able to adjust all of the existing categories as well as delete them. The Admin can adjust any field, aside from the id, and click the `Confirm` button to save the new adjustments.

A category can not be deleted if a non-completed request that uses it exists on the system, and when it is deleted, all requests mentioning it will also be deleted.

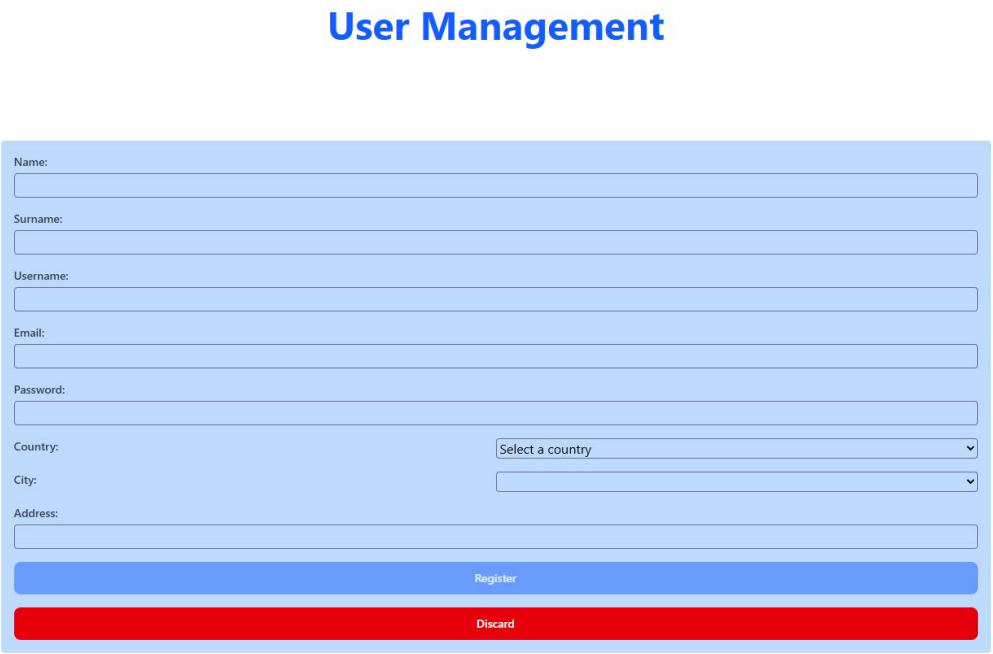
The Admin is also able to create a new request by clicking on the `+ Create New Category` button. This will reveal a form for the Admin to fill out with all the desired information:

The Admin can choose to either click the `Create` button, in order to create the category, or the `Discard` button, in order to throw away the form. Regardless of the choice, the user will be notified through a message about the result:



More importantly, the Admin is also able to manage User information by clicking the `Users` button in the navigation bar, which will take them to the Users Management page:

Here, the Admin is able to find a user by their ID, adjust their information, as well as their role in the system, or delete them. The Admin has also the authority to create a new user by clicking the `+ Register New User` button, which will reveal a form for the Admin to fill with all the desired information:

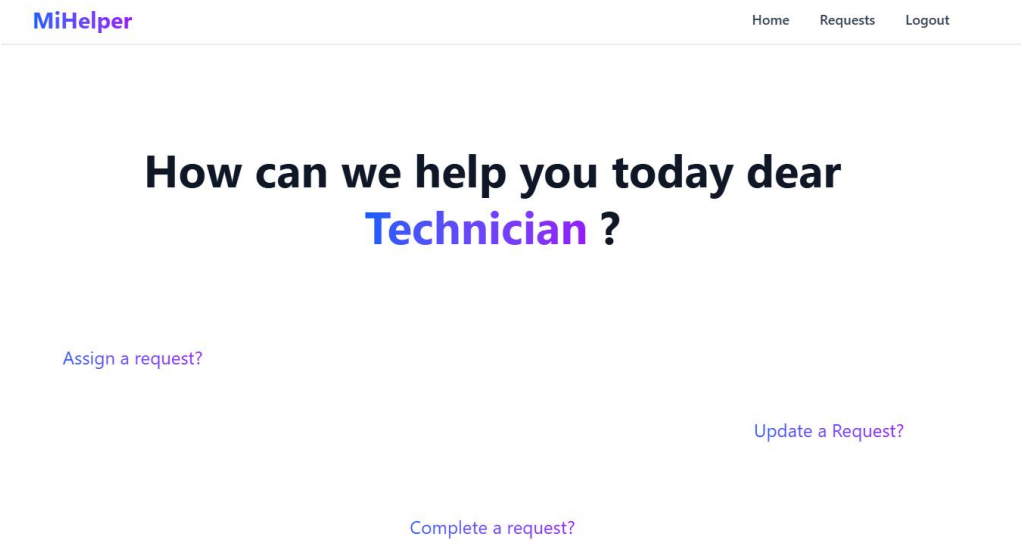


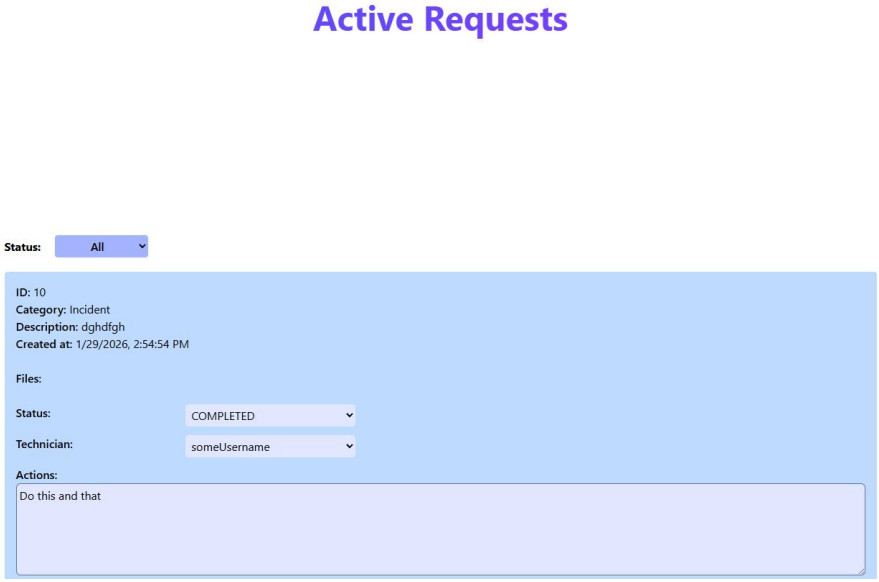
Regardless of the action, the Administrator will be notified accordingly.

Lastly, the Admin is able to logout of the site by clicking the `Logout` button in the navigation bar.

[5.3]: Technician

Once the Technician has logged in with their credentials, they will be shown the Technician Home page:



Here, the Technician is able to choose their next step from the navigation bar, as well as see announcements made by the Admin just like User and the Administrator can.

The main function for the Technicians are the ability to manage all requests in the system, just like the Administrator can: